SKILL-15

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>

int main(void)
{
   int fd;
   int saved_stdout;
   int saved_stderr;

   /* Save the original output streams */
   saved_stdout = dup(STDOUT_FILENO);
   saved_stderr = dup(STDERR_FILENO);

   if (saved_stdout == -1 || saved_stderr == -1) {
      perror("dup");
      if (saved_stdout != -1)
         close(saved_stdout);
      if (saved_stderr != -1)
         close(saved_stderr);
      return 1;
   }

   /* Open a file for combined output */
   fd = open("combined.log",
         O_WRONLY | O_CREAT | O_TRUNC, 0644);
```

```
if (fd == -1) {
    perror("open combined.log");
    close(saved_stdout);
    close(saved_stderr);
    return 1;
}

/* Redirect stdout to the file */
if (dup2(fd, STDOUT_FILENO) == -1) {
    perror("dup2 stdout");
    close(fd);
    close(saved_stdout);
    close(saved_stderr);
    return 1;
}

/* Redirect stderr to the same file */
if (dup2(fd, STDERR_FILENO) == -1) {
    perror("dup2 stderr");
    dup2(saved_stdout, STDOUT_FILENO);
    close(fd);
    close(saved_stdout);
    close(saved_stderr);
    return 1;
}

close(fd);
```

```
printf("1. Standard output message\n");
fflush(stdout);

fprintf(stderr, "2. Standard error message\n");
fflush(stderr);

printf("3. Another standard output message\n");
fflush(stdout);

/* Restore the original streams */
if (dup2(saved_stdout, STDOUT_FILENO) == -1) {
   perror("restore stdout");
   close(saved_stdout);
   close(saved_stderr);
   return 1;
}

if (dup2(saved_stderr, STDERR_FILENO) == -1) {
   perror("restore stderr");
   close(saved_stdout);
   close(saved_stderr);
   return 1;
}

close(saved_stdout);
close(saved_stderr);
```

```
    printf("Redirection completed. Check combined.log\n");


    return 0;
}
```

```
snigdha@Snigdha:~/quotes_practical$ cd ~/quotes_practical
nano combined_redirect.c
snigdha@Snigdha:~/quotes_practical$ gcc -Wall -Wextra -o combined_redirect c
ombined_redirect.c
./combined_redirect
cat combined.log
Redirection completed. Check combined.log
1. Standard output message
2. Standard error message
3. Another standard output message
snigdha@Snigdha:~/quotes_practical$ ls existing_file.txt missing_file.txt >
combined_shell.log 2>&1
cat combined_shell.log
ls: cannot access 'existing_file.txt': No such file or directory
ls: cannot access 'missing_file.txt': No such file or directory
snigdha@Snigdha:~/quotes_practical$ touch existing_file.txt
ls existing_file.txt missing_file.txt > combined_shell.log 2>&1
cat combined_shell.log
ls: cannot access 'missing_file.txt': No such file or directory
existing_file.txt
snigdha@Snigdha:~/quotes_practical$ ls existing_file.txt missing_file.txt >>
 combined_shell.log 2>&1
snigdha@Snigdha:~/quotes_practical$ cd ~/quotes_practical
nano complex_commands.c
snigdha@Snigdha:~/quotes_practical$
```

```
#include <stdio.h>

#include <stdlib.h>

#include <string.h>

#include <unistd.h>

#include <fcntl.h>

#include <sys/wait.h>

#include <errno.h>


#define MAX_STAGES 10

#define MAX_ARGS 64

#define MAX_LINE 1024
```

```
typedef struct {
    char *args[MAX_ARGS];
    int argc;
    char *input_file;
    char *output_file;
    char *error_file;
    int append_output;
} Command;

/* Parse the command into pipeline stages and redirections. */
int parse_command(char *line, Command cmds[], int *stage_count)
{
    char *tokens[MAX_LINE / 2];
    int token_count = 0;
    char *saveptr = NULL;
    char *token;

    memset(cmds, 0, sizeof(Command) * MAX_STAGES);
    *stage_count = 1;

    token = strtok_r(line, " \t\r\n", &saveptr);

    while (token != NULL) {
        if (token_count >= MAX_LINE / 2) {
            fprintf(stderr, "Syntax error: too many tokens.\n");
            return -1;
        }
```

```
    tokens[token_count++] = token;
    token = strtok_r(NULL, " \t\r\n", &saveptr);
}

if (token_count == 0)
    return 0;

for (int i = 0; i < token_count; i++) {
    Command *cmd = &cmds[*stage_count - 1];
    char *t = tokens[i];

    if (strcmp(t, "|") == 0) {
        if (cmd->argc == 0) {
            fprintf(stderr, "Syntax error: empty command before pipe.\n");
            return -1;
        }

        if (*stage_count >= MAX_STAGES) {
            fprintf(stderr, "Syntax error: too many pipeline stages.\n");
            return -1;
        }

        (*stage_count)++;
        continue;
    }

    if (strcmp(t, "<") == 0 || strcmp(t, ">") == 0 ||
        strcmp(t, ">>") == 0 || strcmp(t, "2>") == 0) {
```

```
    if (i + 1 >= token_count ||
        strcmp(tokens[i + 1], "|") == 0 ||
        strcmp(tokens[i + 1], "<") == 0 ||
        strcmp(tokens[i + 1], ">") == 0 ||
        strcmp(tokens[i + 1], ">>") == 0 ||
        strcmp(tokens[i + 1], "2>") == 0) {
        fprintf(stderr,
                "Syntax error: '%s' needs a filename.\n", t);
        return -1;
    }

    char *filename = tokens[++i];

    if (strcmp(t, "<") == 0) {
        cmd->input_file = filename;
    } else if (strcmp(t, ">") == 0) {
        cmd->output_file = filename;
        cmd->append_output = 0;
    } else if (strcmp(t, ">>") == 0) {
        cmd->output_file = filename;
        cmd->append_output = 1;
    } else {
        cmd->error_file = filename;
    }

    continue;
}
```

```
            if (cmd->argc >= MAX_ARGS - 1) {
                fprintf(stderr, "Syntax error: too many arguments.\n");
                return -1;
            }

            cmd->args[cmd->argc++] = t;
        }

        if (cmds[*stage_count - 1].argc == 0) {
            fprintf(stderr, "Syntax error: command missing after pipe.\n");
            return -1;
        }

        for (int i = 0; i < *stage_count; i++) {
            if (cmds[i].argc == 0) {
                fprintf(stderr, "Syntax error: empty pipeline stage.\n");
                return -1;
            }

            cmds[i].args[cmds[i].argc] = NULL;
        }

        return 1;
    }

    /* Apply file redirections using open() and dup2(). */
    int apply_redirections(Command *cmd)
```

```
{
    int fd;

    if (cmd->input_file != NULL) {
        fd = open(cmd->input_file, O_RDONLY);

        if (fd == -1) {
            perror(cmd->input_file);
            return -1;
        }

        if (dup2(fd, STDIN_FILENO) == -1) {
            perror("dup2 input");
            close(fd);
            return -1;
        }

        close(fd);
    }

    if (cmd->output_file != NULL) {
        int flags = O_WRONLY | O_CREAT |
                (cmd->append_output ? O_APPEND : O_TRUNC);

        fd = open(cmd->output_file, flags, 0644);

        if (fd == -1) {
            perror(cmd->output_file);
```

```
            return -1;
        }

        if (dup2(fd, STDOUT_FILENO) == -1) {
            perror("dup2 output");
            close(fd);
            return -1;
        }

        close(fd);
    }

    if (cmd->error_file != NULL) {
        fd = open(cmd->error_file,
                O_WRONLY | O_CREAT | O_TRUNC, 0644);

        if (fd == -1) {
            perror(cmd->error_file);
            return -1;
        }

        if (dup2(fd, STDERR_FILENO) == -1) {
            perror("dup2 stderr");
            close(fd);
            return -1;
        }

        close(fd);
```

```
    }

    return 0;
}

/* Create pipes, fork processes, redirect streams, and execute commands. */
int execute_pipeline(Command cmds[], int count)
{
    int pipes[MAX_STAGES - 1][2];
    pid_t pids[MAX_STAGES];
    int created = 0;

    for (int i = 0; i < count - 1; i++) {
        if (pipe(pipes[i]) == -1) {
            perror("pipe");

            for (int j = 0; j < i; j++) {
                close(pipes[j][0]);
                close(pipes[j][1]);
            }

            return 1;
        }
    }

    for (int i = 0; i < count; i++) {
        pid_t pid = fork();
```

```
if (pid == -1) {
   perror("fork");
   break;
}

if (pid == 0) {
   /* Connect this command to the previous pipe. */
   if (i > 0 &&
      dup2(pipes[i - 1][0], STDIN_FILENO) == -1) {
      perror("dup2 pipe input");
      _exit(1);
   }

   /* Connect this command to the next pipe. */
   if (i < count - 1 &&
      dup2(pipes[i][1], STDOUT_FILENO) == -1) {
      perror("dup2 pipe output");
      _exit(1);
   }

   for (int j = 0; j < count - 1; j++) {
      close(pipes[j][0]);
      close(pipes[j][1]);
   }

   /* Explicit file redirections override pipe endpoints. */
   if (apply_redirections(&cmds[i]) == -1)
      _exit(1);
```

```
        execvp(cmds[i].args[0], cmds[i].args);

        perror(cmds[i].args[0]);
        _exit(errno == ENOENT ? 127 : 126);
    }

    pids[i] = pid;
    created++;
}

for (int i = 0; i < count - 1; i++) {
    close(pipes[i][0]);
    close(pipes[i][1]);
}

int result = 0;

for (int i = 0; i < created; i++) {
    int status;

    if (waitpid(pids[i], &status, 0) == -1) {
        perror("waitpid");
        result = 1;
    } else if (!WIFEXITED(status) || WEXITSTATUS(status) != 0) {
        result = 1;
    }
}
```

```
    return result;
}

/* Display the parsed execution plan. */
void show_plan(Command cmds[], int count)
{
    printf("Execution plan: %d stage(s)\n", count);

    for (int i = 0; i < count; i++) {
        printf("  Stage %d: ", i + 1);

        for (int j = 0; j < cmds[i].argc; j++)
            printf("%s%s", j ? " " : "", cmds[i].args[j]);

        if (cmds[i].input_file)
            printf(" [stdin <- %s]", cmds[i].input_file);

        if (cmds[i].output_file)
            printf(" [stdout %s %s]",
                cmds[i].append_output ? "append-to" : "write-to",
                cmds[i].output_file);

        if (cmds[i].error_file)
            printf(" [stderr -> %s]", cmds[i].error_file);

        putchar('\n');
    }
```

```
}

int main(void)
{
    char line[MAX_LINE];

    printf("Complex Command Executor\n");
    printf("Supported: |, <, >, >>, 2>\n");
    printf("Use spaces around operators. Type exit to quit.\n");

    while (1) {
        Command commands[MAX_STAGES];
        int stage_count = 0;

        printf("complex-shell> ");
        fflush(stdout);

        if (fgets(line, sizeof(line), stdin) == NULL) {
            putchar('\n');
            break;
        }

        if (strcmp(line, "exit\n") == 0 ||
            strcmp(line, "exit") == 0)
            break;

        int parsed = parse_command(line, commands, &stage_count);
```

```
    if (parsed < 0)
        continue;

    if (parsed == 0)
        continue;

    show_plan(commands, stage_count);
    execute_pipeline(commands, stage_count);
  }

  return 0;
}
```

